

CONFIGURATION_SYSTEM

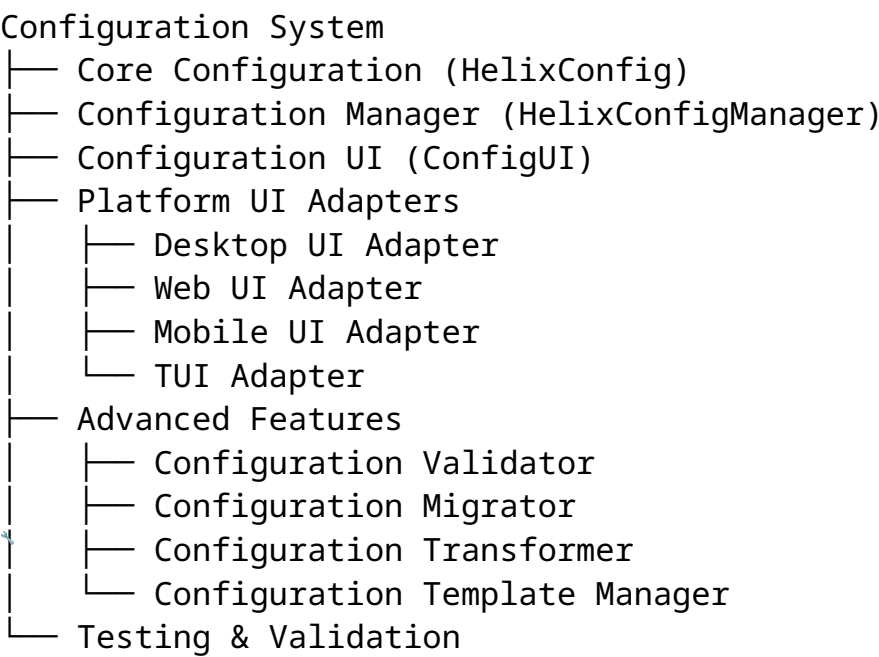
HelixCode Configuration System - Complete Guide

Overview

The HelixCode configuration system provides a comprehensive, platform-optimized, and extensible framework for managing application settings across all supported platforms. It supports real-time validation, migration, templating, and platform-specific UI components.

Architecture

Core Components



Core Configuration Structure

Application Configuration

```
application:
  name: "HelixCode"
  description: "Distributed AI Development Platform"
```

```
version: "1.0.0"
environment: "development" # development, testing, staging,
                        production
workspace:
  default_path: "~/helixcode"
  auto_save: true
  backup_enabled: true
  sync_interval: "5m"
sessions:
  auto_save: true
  save_interval: "30s"
  max_sessions: 10
  encryption_enabled: true
logging:
  level: "info" # debug, info, warn, error
  format: "structured" # structured, text, json
  output: "both" # file, console, both
  file_path: "~/helixcode/logs/helix.log"
  max_file_size: "100MB"
  max_files: 10
  module_levels: {}
  structured_logging: true
  correlation_id: true
  stack_traces: true
  log_performance: true
  log_slow_queries: true
  slow_query_threshold: "1s"
ui:
  theme: "dark" # dark, light, auto
  language: "en" # en, zh, es, fr, de, ja
  font_size: 14
  font_family: "SF Mono"
  sidebar_width: 250
  status_bar_visible: true
  toolbar_visible: true
  tab_bar_visible: true
  minimap_enabled: true
  line_numbers_enabled: true
  word_wrap_enabled: true
  auto_complete_enabled: true
  auto_indent_enabled: true
  bracket_matching_enabled: true
  cursor_blink: true
  smooth_scrolling: true
```

```
animations_enabled: true
hot_reload: true
accessibility:
  screen_reader: true
  high_contrast: false
  keyboard_navigation: true
  focus_visible: true
  reduced_motion: false
  font_scaling: 1.0
  icon_scaling: 1.0
  color_blind_mode:
    "none" # none, protanopia, deuteranopia, tritanopia
shortcuts:
  custom_shortcuts: {}
  vi_mode: false
  emacs_mode: false
```

Database Configuration

```
database:
  type: "postgresql" # postgresql, mysql, sqlite
  host: "localhost"
  port: 5432
  database: "helixcode"
  username: "helixcode"
  password: ""
  ssl_mode: "require" # disable, allow, prefer, require, verify-
    ca, verify-full
connection_pool:
  max_connections: 20
  max_idle_connections: 10
  connection_lifetime: "1h"
  health_check_interval: "30s"
  health_check_timeout: "5s"
backup:
  enabled: true
  schedule: "0 2 * * *" # Daily at 2 AM
  retention: "30d"
  encryption_enabled: true
  compression_enabled: true
migrations:
  auto_migrate: true
  migration_timeout: "5m"
  backup_before_migrate: true
```

Redis Configuration

```
redis:
  enabled: true
  host: "localhost"
  port: 6379
  password: ""
  database: 0
  connection_pool:
    max_connections: 10
    max_idle_connections: 5
    connection_lifetime: "30m"
    health_check_interval: "10s"
  security:
    tls_enabled: false
    cert_file: ""
    key_file: ""
    ca_cert: ""
  clustering:
    enabled: false
    nodes: []
    sentinel_enabled: false
    sentinel_master: ""
    sentinel_nodes: []
```

LLM Configuration

```
llm:
  default_provider: "local" # local, openai, anthropic, gemini,
                             qwen, azure, vertexai, bedrock, xai, groq, openrouter,
                             copilot
  default_model: "llama-3.2-3b"
  providers:
    openai:
      api_key: ""
      base_url: "https://api.openai.com/v1"
      organization: ""
      timeout: "30s"
      max_retries: 3
      models: ["gpt-4", "gpt-4-turbo", "gpt-3.5-turbo"]
    anthropic:
      api_key: ""
      base_url: "https://api.anthropic.com"
      timeout: "30s"
```

```
    max_retries: 3
    models: ["claude-3-opus", "claude-3-sonnet", "claude-3-haiku"]
gemini:
    api_key: ""
    base_url: "https://generativelanguage.googleapis.com/v1beta"
    timeout: "30s"
    max_retries: 3
    models: ["gemini-pro", "gemini-pro-vision"]
qwen:
    api_key: ""
    base_url: "https://dashscope.aliyuncs.com/api/v1"
    timeout: "30s"
    max_retries: 3
    models: ["qwen-turbo", "qwen-plus", "qwen-max"]
azure:
    api_key: ""
    endpoint: ""
    api_version: "2024-02-15-preview"
    timeout: "30s"
    max_retries: 3
    deployment_names: {}
vertexai:
    service_account_key: ""
    project_id: ""
    location: "us-central1"
    timeout: "30s"
    max_retries: 3
bedrock:
    access_key_id: ""
    secret_access_key: ""
    region: "us-east-1"
    timeout: "30s"
    max_retries: 3
xai:
    api_key: ""
    base_url: "https://api.x.ai/v1"
    timeout: "30s"
    max_retries: 3
groq:
    api_key: ""
    base_url: "https://api.groq.com/openai/v1"
    timeout: "30s"
```

```

    max_retries: 3
openrouter:
  api_key: ""
  base_url: "https://openrouter.ai/api/v1"
  timeout: "30s"
  max_retries: 3
copilot:
  token: ""
  base_url: "https://api.githubcopilot.com"
  timeout: "30s"
  max_retries: 3
features:
  reasoning_enabled: true
  streaming_enabled: true
  function_calling_enabled: true
  image_generation_enabled: true
  audio_transcription_enabled: false
  code_execution_enabled: false
  web_search_enabled: false
optimization:
  cost_management:
    budget_enabled: false
    monthly_budget: 100.0
    cost_tracking: true
    cost_alerts: true
  token_management:
    max_tokens_per_request: 4096
    context_window_size: 8192
    token_usage_tracking: true
  response_caching:
    enabled: true
    cache_ttl: "1h"
    cache_size: "1GB"
  model_fallback:
    enabled: true
    fallback_providers: ["local"]
    fallback_models: ["llama-3.2-3b"]
    quality_threshold: 0.8

```

Platform Configuration

```

platform:
  current_platform: "desktop" # desktop, web, mobile, tui
desktop:

```

```
enabled: true
auto_start: false
minimize_to_tray: true
show_in_taskbar: true
file_associations: [".go", ".js", ".py", ".java", ".cpp",
                    ".h"]
context_menu: true
auto_update: true
hardware_acceleration: true
gpu_acceleration: true
memory_limit: 2147483648 # 2GB
ui_scale: 1.0
high_dpi: true
web:
  enabled: true
  host: "localhost"
  port: 3000
  base_path: "/"
  static_path: "./static"
  cache_enabled: true
  cache_ttl: 3600
  pwa_enabled: true
  offline_enabled: true
  cs_protection: true
  xss_protection: true
  compression_enabled: true
  minify_enabled: true
  real_time_updates: true
  websocket_enabled: true
mobile:
  enabled: true
  ios:
    enabled: true
    app_store_connect: false
    push_notifications: false
    background_tasks: true
    watch_kit_app: false
    team_id: ""
    bundle_id: "dev.helix.code"
    development_cert: false
  android:
    enabled: true
    google_play_console: false
    push_notifications: false
```

```
background_tasks: true
wear_os_app: false
package_name: "dev.helix.code"
signing_enabled: true
debug_build: true
cross_platform:
  framework: "react-native"
  offline_first: true
  sync_enabled: true
  image_optimization: true
  lazy_loading: true
  biometric_auth: true
  device_encryption: true
tui:
  enabled: true
  compatibility_mode: "auto"
  color_scheme: "dark"
  true_color: true
  mouse_enabled: true
  render_fps: 60
  buffer_size: 10000
  status_line: true
  tab_bar: true
  split_screen: true
aurora_os:
  enabled: false
  sailfish_integration: true
  store_integration: false
  sdk_version: "4.4"
  native_ui: true
  qt_components: true
harmony_os:
  enabled: false
  harmony_services: true
  app_gallery: false
  dev_eco_studio: false
  sdk_version: "5.0"
  ark_ui: true
cross_platform:
  consistent_theme: true
  sync_config: true
  sync_data: true
  update_across_platforms: true
```



```
common_feature_set: true  
platform_optimizations: true
```

Platform-Specific UI

Desktop UI Features

- **Native Window Management:** Resizable, minimizable, full-screen support
- **System Tray Integration:** Minimize to system tray with background operation
- **Native Menus:** System-native menu bar and context menus
- **File Associations:** Automatic file type associations and registration
- **Native Fonts:** System font rendering and anti-aliasing
- **Keyboard Shortcuts:** Platform-specific keyboard shortcuts
- **Drag & Drop:** Native drag and drop support
- **System Notifications:** Native desktop notification support
- **Auto Updates:** Automatic update checking and installation
- **GPU Acceleration:** Hardware-accelerated rendering

Web UI Features

- **Responsive Design:** Adaptive layouts for all screen sizes
- **PWA Support:** Progressive Web App capabilities
- **Offline Support:** Service worker-based offline functionality
- **WebSockets:** Real-time bidirectional communication
- **Touch Support:** Touch-friendly interface for mobile browsers
- **Browser Storage:** LocalStorage and IndexedDB integration
- **Push Notifications:** Web Push API support
- **CSS Animations:** Smooth transitions and micro-interactions
- **Compression:** Gzip/Brotli compression for performance
- **Accessibility:** WCAG 2.1 AA compliance

Mobile UI Features

- **Touch Gestures:** Swipe, pinch, tap, long-press support
- **Biometric Authentication:** Face ID, Touch ID, fingerprint support
- **Push Notifications:** Native push notification handling
- **Offline First:** Local-first data synchronization
- **App Lifecycle:** Proper background/foreground handling
- **Camera Access:** Device camera integration
- **Location Services:** GPS and location-based features
- **Device Orientation:** Auto-rotation and orientation locking
- **Native Plugins:** Access to device hardware and APIs
- **Battery Optimization:** Efficient power usage

TUI Features

- **Terminal Colors:** ANSI color support and themes
- **Keyboard Navigation:** Full keyboard accessibility
- **Mouse Support:** Mouse interaction in terminal
- **Unicode Support:** International character support
- **Screen Reader:** Compatibility with screen readers
- **Clipboard Access:** Terminal clipboard integration
- **Terminal Shortcuts:** Common terminal keybindings
- **Resize Handling:** Dynamic terminal resize support
- **Buffer Management:** Large file and output handling
- **Performance:** Optimized rendering and scrolling

Validation System

Built-in Validation Rules

- **Type Validation:** String, number, boolean, array, object validation
- **Range Validation:** Min/max values for numbers
- **Length Validation:** Min/max lengths for strings
- **Pattern Validation:** Regex pattern matching
- **Enum Validation:** Value must be in allowed enum
- **Format Validation:** Email, URL, date-time, IPv4, hostname
- **Required Fields:** Required field validation
- **Custom Rules:** Extensible custom validation functions

Validation Examples

```
// Basic validation
validator := NewConfigurationValidator(true)
config := getDefaultConfig()
result := validator.Validate(config)

// Check validation result
if !result.Valid {
    for _, err := range result.Errors {
        fmt.Printf("Error: %s\n", err.Message)
    }
}

// Custom validation rule
validator.AddCustomRule("database.password", func(value
    interface{}) error {
    if str, ok := value.(string); ok {
```

```

        if len(str) < 8 {
            return fmt.Errorf("password must be at least 8
characters")
        }
    }
    return nil
})

```

Migration System

Migration Features

- **Version Tracking:** Configuration version management
- **Automatic Migration:** Seamless version upgrades
- **Backup Support:** Automatic backups before migration
- **Rollback Support:** Migration rollback capability
- **Path Finding:** Intelligent migration path calculation
- **Dry Run:** Test migrations without applying changes

Migration Example

```

migrator := NewConfigurationMigrator("1.0.0")

// Define migration
migration := Migration{
    From:    "1.0.0",
    To:      "1.1.0",
    Name:    "add_workspace_auto_save",
    Desc:    "Add workspace auto_save setting",
    Up: func(config *HelixConfig) error {
        config.Application.Workspace.AutoSave = true
        return nil
    },
    Backup: true,
}

// Apply migration
err := migrator.Migrate(config, "1.2.0")

```

Template System

Template Features

- **Parameterized Templates:** Variables and substitutions

- **Variable Validation:** Type checking and constraints
- **Default Environments:** Development, testing, production templates
- **Custom Templates:** User-defined templates
- **Template Inheritance:** Template composition and extension
- **Export/Import:** Template sharing and backup

Template Example

```
# development-template.yaml
id: "development"
name: "Development Environment"
description: "Configuration optimized for development"
category: "environment"
author: "system"
version: "1.0.0"
tags: ["development", "debug"]
variables:
  workspace_path:
    name: "Workspace Path"
    type: "string"
    default: "~/development/helixcode"
    required: true
    pattern: "^[^/].*"
  debug_enabled:
    name: "Debug Enabled"
    type: "boolean"
    default: true
    required: false
config:
  application:
    environment: "development"
    workspace:
      auto_save: true
      default_path: "{{workspace_path}}"
  development:
    enabled: true
    debug:
      enabled: "{{debug_enabled}}"
      level: "debug"
```

Security Features

Security Configuration

```
security:
  authentication:
    enabled: true
    methods: ["password", "2fa", "oauth", "certificate"]
    password_policy:
      min_length: 8
      require_uppercase: true
      require_lowercase: true
      require_numbers: true
      require_special: true
    session_management:
      timeout: "30m"
      max_concurrent_sessions: 3
      remember_me_duration: "30d"
    oauth_providers: ["github", "google", "microsoft"]
  authorization:
    enabled: true
    rbac: true
    roles: ["admin", "developer", "viewer"]
    permissions: {}
    default_policy: "deny"
  encryption:
    enabled: true
    algorithm: "AES-256-GCM"
    key_rotation: "90d"
    at_rest: true
    in_transit: true
  data_protection:
    encryption_at_rest: true
    encryption_in_transit: true
    key_rotation: "90d"
    retention_policy:
      enabled: true
      default_retention: "365d"
      specific_retention: {}
      auto_delete: false
      notification_period: "30d"
  masking_enabled: false
  masked_fields: []
  backup_encryption: true
```

```
network:
  firewall_enabled: false
  allowed_ips: []
  blocked_ips: []
  allowed_ports: [8080, 22]
  blocked_ports: []
  tls_enabled: false
  tls_version: "1.3"
  cipher_suites: []
  vpn_enabled: false
  vpn_provider: ""
  vpn_config: {}
audit:
  enabled: true
  log_level: "info"
  log_path: "~/.helixcode/logs/audit.log"
  max_log_size: "100MB"
  log_retention: 90
  events: ["login", "logout", "config_change",
           "task_execution"]
  real_time_enabled: false
  alert_endpoints: []
privacy:
  data_collection_enabled: false
  analytics_enabled: false
  consent_required: true
  consent_version: "1.0"
  data_sharing_enabled: false
  shared_data_types: []
  anonymize_data: true
  anonymization_method: "hashing"
  right_to_deletion: true
  data_deletion_method: "secure_erase"
```

Performance Optimization

Configuration Performance

- **Lazy Loading:** Load configuration sections on demand
- **Caching:** In-memory configuration caching
- **Validation Caching:** Cached validation results
- **Batch Updates:** Atomic configuration updates
- **Change Detection:** Efficient change detection
- **Memory Optimization:** Minimal memory footprint

Performance Metrics

```
performance:
  optimization:
    enabled: true
    auto_tune: true
    metrics_collection: true
  profiling:
    enabled: false
    sampling_rate: 100.0
    max_duration: "5m"
    auto_analysis: true
    analysis_tool: "go tool pprof"
  caching:
    config_cache: true
    validation_cache: true
    template_cache: true
    cache_ttl: "1h"
    max_cache_size: "100MB"
  monitoring:
    enabled: true
    metrics_interval: "1m"
    alert_thresholds: {}
    dashboard_enabled: true
```

Usage Examples

Basic Configuration Usage

```
// Load configuration
config, err := LoadHelixConfig()
if err != nil {
    log.Fatal(err)
}

// Update configuration
UpdateHelixConfig(func(config *HelixConfig) {
    config.LLM.DefaultProvider = "openai"
    config.Server.Port = 9090
    config.Application.Logging.Level = "debug"
})

// Save configuration
if err := SaveHelixConfig(config); err != nil {
```

```
    log.Fatal(err)
}
```

Platform UI Usage

```
// Get appropriate UI adapter for current platform
adapter := GetPlatformUIAdapterForCurrentPlatform()

// Create configuration UI
configUI, err := NewConfigUI(configPath)
if err != nil {
    log.Fatal(err)
}

// Render configuration form
form, err := adapter.RenderConfigForm(configUI)
if err != nil {
    log.Fatal(err)
}

// Show configuration dialog
changesMade, err := adapter.ShowConfigDialog(configUI)
if err != nil {
    log.Fatal(err)
}

if changesMade {
    fmt.Println("Configuration was modified")
}
```

Template Usage

```
// Create template manager
manager := NewConfigurationTemplateManager()

// Apply development template
variables := map[string]interface{}{
    "workspace_path": "~/my_project",
    "debug_enabled":  true,
}

config, err := manager.ApplyTemplate("development", variables)
if err != nil {
    log.Fatal(err)
}
```

```

}

// Create custom template
template, err := manager.CreateTemplateFromConfig(config, "My
    Template", "Custom configuration", nil)
if err != nil {
    log.Fatal(err)
}

```

Validation Usage

```

// Create validator
validator := NewConfigurationValidator(true)

// Add custom validation
validator.AddCustomRule("database.host", func(value interface{})
    error {
        if host, ok := value.(string); ok {
            if host == "" {
                return fmt.Errorf("database host cannot be empty")
            }
        }
        return nil
    })

// Validate configuration
result := validator.Validate(config)
if !result.Valid {
    fmt.Println("Configuration validation failed:")
    for _, err := range result.Errors {
        fmt.Printf("  %s: %s\n", err.Path, err.Message)
    }
}

```

Advanced Features

Configuration Transformation

- **Field Mapping:** Copy, move, rename configuration fields
- **Type Conversion:** Automatic type conversion between fields
- **Conditional Logic:** Apply transformations based on conditions
- **Template Substitution:** Variable substitution in configuration values
- **Calculation:** Dynamic value calculation

Configuration Watchers

- **Real-time Monitoring:** Monitor configuration changes
- **Change Notifications:** Notify on configuration updates
- **Validation on Change:** Validate changes before applying
- **Rollback on Error:** Automatic rollback on validation failures

Configuration Snapshots

- **Version History:** Track configuration changes over time
- **Named Snapshots:** Create named configuration snapshots
- **Comparison:** Compare configuration versions
- **Restore:** Restore previous configurations

Best Practices

Configuration Design

1. **Default Values:** Provide sensible defaults for all settings
2. **Validation:** Validate all configuration values
3. **Migration:** Support seamless configuration upgrades
4. **Documentation:** Document all configuration options
5. **Testing:** Test all configuration scenarios

Security

1. **Sensitive Data:** Never log passwords or API keys
2. **Encryption:** Encrypt sensitive configuration values
3. **Access Control:** Restrict configuration access
4. **Audit Trail:** Log all configuration changes
5. **Backups:** Regular configuration backups

Performance

1. **Lazy Loading:** Load configuration sections on demand
2. **Caching:** Cache frequently accessed configuration
3. **Validation:** Optimize validation performance
4. **Updates:** Use atomic updates for consistency
5. **Monitoring:** Monitor configuration performance

Integration

API Integration

// Configuration API endpoints

GET	/api/config	- Get current configuration
POST	/api/config	- Update configuration
PUT	/api/config/{section}	- Update configuration section
DELETE	/api/config/{section}	- Reset configuration section
POST	/api/config/validate	- Validate configuration
GET	/api/config/schema	- Get configuration schema
POST	/api/config/backup	- Create configuration backup
POST	/api/config/restore	- Restore configuration
GET	/api/config/templates	- List available templates
POST	/api/config/templates	- Apply template

CLI Integration

Configuration CLI commands

helix config show	- Show current configuration
helix config get <key> value	- Get configuration
helix config set <key> <value> value	- Set configuration
helix config reset <section> section	- Reset configuration
helix config validate configuration	- Validate
helix config backup	- Create backup
helix config restore <file>	- Restore from backup
helix config template list	- List templates
helix config template apply <name>	- Apply template
helix config migrate <to-version> configuration	- Migrate

References

Configuration Schema

- [Complete Schema Reference](#)
- [Validation Rules](#)
- [Migration Guide](#)
- [Template Documentation](#)

Platform Guides

- [Desktop Configuration Guide](#)
- [Web Configuration Guide](#)
- [Mobile Configuration Guide](#)
- [TUI Configuration Guide](#)

Development

- [Configuration API](#)
- [Plugin Development](#)
- [Custom Validation](#)
- [Testing Configuration](#)

This configuration system provides a solid foundation for managing HelixCode across all platforms, with comprehensive features for validation, migration, templating, and platform optimization.